

1. URL del repositorio de control de versiones usado

El repositorio de control de versiones del proyecto GymTrack estará alojado en GitHub.

URL del repositorio:

<https://github.com/lmoyansa/gymtrack>

El repositorio se utilizará para almacenar el código fuente de la aplicación Android, la documentación principal del proyecto y el histórico de cambios mediante commits.

Además, se utilizará Notion como herramienta de gestión ágil para organizar tareas, historias de usuario, sprints, responsables simulados y seguimiento del proyecto.

2. Justificación de la plataforma usada

Para el control de versiones se ha elegido **GitHub**, ya que permite trabajar con Git, mantener un historial de cambios mediante commits y organizar el desarrollo mediante ramas.

Aunque el proyecto se desarrolla de forma individual, GitHub permite simular una dinámica profesional de trabajo, separando el desarrollo de funcionalidades, la documentación y las versiones estables del proyecto.

Para la gestión ágil del proyecto se utilizará **Notion**, ya que permite organizar el trabajo mediante un tablero visual con tareas, estados, prioridades, sprints y responsables. En Notion se llevará el seguimiento del Product Backlog, las tareas del Sprint Backlog, las reuniones con el profesor y el avance del proyecto.

El desarrollo técnico de la aplicación se realizará en **Android Studio**, ya que GymTrack será una aplicación móvil Android. Android Studio permite crear la interfaz de usuario, implementar la lógica de la aplicación, gestionar recursos y probar la app en emulador o dispositivo físico.

Por tanto, se utilizarán tres herramientas principales:

- **GitHub**: repositorio de código, documentación y control de versiones.
- **Notion**: gestión ágil del proyecto, tareas, sprints y reuniones.
- **Android Studio**: desarrollo de la aplicación Android.

3. Distribución de permisos de los usuarios

El proyecto será desarrollado de forma individual por Luis Moyano, por lo que el permiso real sobre el repositorio será de administrador.

Aun así, para simular un entorno de trabajo ágil, se diferenciarán varios perfiles con responsabilidades concretas dentro del proyecto.

Permisos reales

- **Luis Moyano:** administrador del repositorio de GitHub y del espacio de trabajo de Notion.
- **Profesor:** acceso de revisión al repositorio y a la documentación, según se solicite durante las reuniones de seguimiento.

Perfiles simulados

Jefe de proyecto / Product Owner

Responsable de definir la visión del producto, priorizar funcionalidades y decidir el alcance del MVP.

Scrum Master

Responsable de organizar el seguimiento del proyecto, mantener actualizado el tablero de Notion, preparar reuniones y detectar bloqueos.

Analista

Responsable de revisar historias de usuario, definir tareas y subtareas, analizar el modelo de datos y comprobar que el prototipo cubre las funcionalidades previstas.

Desarrollador Android

Responsable de implementar la aplicación en Android Studio, crear pantallas, modelos y funcionalidades.

Responsable de repositorio

Responsable de gestionar ramas, commits, estructura del repositorio y documentación técnica asociada.

Tester / QA

Responsable de definir y ejecutar pruebas básicas para validar que las funcionalidades cumplen los criterios de aceptación.

Documentador

Responsable de mantener la documentación del proyecto, actas de reuniones, documentación de sprints y documentos entregables.

4. Política de actualización de código

Para mantener el repositorio organizado se seguirá una política sencilla de ramas y commits.

4.1 Política de ramas

Se utilizarán las siguientes ramas:

main

Será la rama principal del repositorio. Contendrá únicamente versiones estables del proyecto y documentación final revisada.

develop

Será la rama de desarrollo. Se utilizará para implementar y probar la aplicación Android antes de pasar los cambios estables a main.

documentation

Será la rama destinada a la documentación del proyecto. Incluirá documentos como la propuesta inicial, prototipo, actas de reuniones, documentación de sprints, modelo de datos, estrategia de pruebas y documentos de entrega.

feature/nombre-funcionalidad

Se podrán utilizar ramas de funcionalidad para desarrollar partes concretas de la aplicación, en caso de que sea necesario.

Ejemplos:

feature/login
feature/crear-rutina
feature/anadir-ejercicio
feature/registro-entrenamiento
feature/historial
feature/cronometro

Flujo de trabajo previsto

1. Las tareas se planifican en Notion.
2. Se trabaja sobre la rama correspondiente según el tipo de tarea.
3. Los cambios de código se realizan en develop o en ramas feature.
4. La documentación se trabaja en la rama documentacion.
5. Cuando una parte esté revisada y estable, podrá integrarse en main.
6. La rama main representará siempre una versión estable del proyecto.

5. Política de commits

Los commits deberán ser claros, concretos y descriptivos.

Cada commit representará un cambio concreto del proyecto. Se evitarán commits demasiado grandes o que mezclen varias funcionalidades distintas.

Normas principales

- Los commits deben describir claramente el cambio realizado.
- Se evitará subir cambios incompletos o que impidan compilar el proyecto.
- Siempre que sea posible, los commits estarán relacionados con tareas del tablero de Notion.
- La rama main solo recibirá cambios estables.
- La documentación también se versionará mediante commits.

Ejemplos de commits

Inicializa proyecto Android
Añade README inicial
Crea carpeta docs
Añade prototipo inicial
Define política de ramas
Define política de commits
Crea modelo Rutina
Crea modelo Ejercicio
Implementa pantalla de listado de rutinas
Implementa creación de rutina
Añade cronómetro de descanso
Corrige validación de rutina sin nombre
Actualiza documentación del Sprint 1

6. Estructura inicial del código fuente

La estructura inicial del repositorio será la siguiente:

```
gymtrack/
├── README.md
├── .gitignore
├── docs/
│   ├── propuesta-proyecto.pdf
│   ├── prototipo.pdf
│   ├── gestion-repositorio.pdf
│   ├── reuniones/
│   │   └── reunion-01-planificacion.md
│   ├── sprints/
│   │   ├── sprint-0.md
│   │   ├── sprint-1.md
│   │   └── sprint-2.md
├── app/
│   ├── build.gradle
│   └── src/
│       ├── main/
│       │   ├── AndroidManifest.xml
│       │   └── java/com/example/gymtrack/
│       │       ├── ui/
│       │       ├── model/
│       │       ├── data/
│       │       ├── repository/
│       │       └── utils/
│       └── res/
│           ├── drawable/
│           ├── layout/
│           ├── mipmap/
│           └── values/
└── test/
    └── androidTest/
```

Descripción de carpetas

[README.md](#)

Archivo principal del repositorio. Incluye descripción del proyecto, tecnologías utilizadas, estructura inicial e instrucciones básicas de ejecución.

docs/

Carpeta destinada a la documentación del proyecto: propuesta inicial, prototipo, gestión del repositorio, actas de reuniones y documentación de sprints.

docs/reuniones/

Carpeta destinada a guardar las actas de reuniones con el profesor.

docs/sprints/

Carpeta destinada a documentar el objetivo, tareas principales, estado y resultado de cada sprint.

app/

Carpeta principal de la aplicación Android.

ui/

Contendrá las pantallas y componentes visuales de la aplicación.

model/

Contendrá las clases principales del modelo de datos, como Usuario, Rutina, Ejercicio y Entrenamiento.

data/

Contendrá la lógica relacionada con el almacenamiento de datos.

repository/

Contendrá la capa intermedia entre la interfaz de usuario y la capa de datos.

utils/

Contendrá utilidades generales, validaciones y funciones auxiliares, como el cronómetro.

test/

Contendrá pruebas unitarias.

androidTest/

Contendrá pruebas instrumentadas de Android.

7. Relación con la metodología ágil

El proyecto se organizará en tres sprints:

Sprint 0

Propuesta inicial, definición del MVP, primeras historias de usuario y tablero de tareas.

Sprint 1

Análisis, revisión de historias de usuario, definición de tareas y subtareas, prototipo, flujo de navegación, modelo de datos, estrategia de pruebas, repositorio y documentación inicial.

Sprint 2

Implementación de la aplicación, pruebas básicas, corrección de errores y documentación final.

Notion se utilizará para gestionar el seguimiento diario del proyecto, mientras que GitHub almacenará el código fuente y la documentación versionada.

8. Conclusión

La organización planteada permite separar claramente la gestión ágil del proyecto, el desarrollo técnico de la aplicación y la documentación asociada.

GitHub funcionará como repositorio oficial del proyecto, Notion como herramienta de gestión ágil y Android Studio como entorno de desarrollo. Aunque el proyecto sea individual, se simularán distintos perfiles de trabajo y se documentará el avance mediante tareas, subtareas, sprints, commits y documentación versionada.